Inženjerstvo serverskog sloja – Vežbe 9
Spring Security

Spring Security je radni okvir koji pruža podršku za autentifikaciju i autorizaciju korisnika, kao i dodatne mehanizme zaštite od određenih napada na web aplikacije. Spring Security se u suštini zasniva na upotrebi Servlet Filtera, koji presreću zahteve koji stižu od korisnika i odlučuju da li zahtev treba da se odbije ili prosledi dalje u aplikaciju, do kontrolera. Može da se integriše i sa običnom Spring aplikacijom, kao i sa Spring Boot i Spring MVC radnim okvirom.

Pre nego što krenete sa integracijom Spring Security-ja sa aplikacijom, neophodno je razumeti tri osnovna koncepta:

- Autentifikacija
- Autorizacija
- Filteri

Autentifikacija

Autentifikacija je proces koji se sastoji od utvrđivanja da li je korisnik poznat sistemu (identifikacija) i potvrđivanja da je korisnik stvarno onaj koji tvrdi da jeste. Postoje različiti mehanizmi za autentifikaciju, ali se najčešće koristi korisničko ime i lozinka. Korisnik unosi svoje korisničko ime i lozinku koji se u paru šalju na serversku stranu. Server zatim provera da li je korisničko ime poznato sistemu, i ako jeste, proverava se da li se lozinka poklapa sa onim što je zapisano u bazi.

Autorizacija

U jednostavnijim aplikacijama autentifikacija može biti dovoljna: čim korisnik potvrdi svoj identitet, može pristupiti svakom delu aplikacije.

Ali u većini aplikacija postoje koncepti permisija (dozvola) ili uloga. Razmotrimo primer web shop aplikacije. Postoje kupci koji imaju pristup javnom delu web shop-a, gde mogu da pretražuju i kupuju proizvode. Sa druge strane, postoje i administratori koji imaju pristup zasebnom administrativnom delu aplikacije, gde mogu da unose nove proizvode i ažuriraju cene.

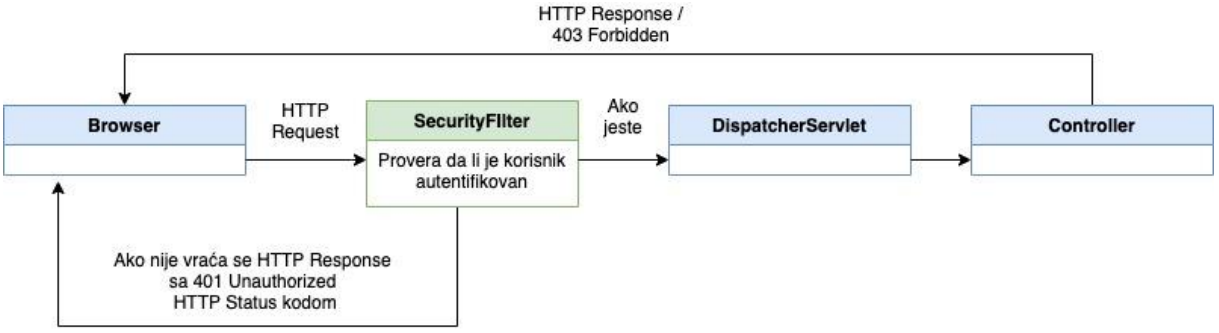
Oba tipa korisnika se moraju autentifikovati, ali sama činjenica da je korisnik poznat sistemu ne govori ništa o tome šta im je dozvoljeno da rade u sistemu. Stoga se moraju proveriti i uloge ili permisije autentifikovanog korisnika, tj. treba proći kroz proces autorizacije.

Filteri

U prethodnom primeru objašnjen je koncept DispatcherServlet-a koji redirektuje svaki HTTP Request (na primer, koji šalje Browser) do odgovarajućih kontrolera. To znači da

svaku Spring aplikaciju možemo posmatrati kao samo jedan Servlet. Stvar je u tome što DispatcherServlet nema implementirane bezbednosne mehanizme, a verovatno nećete želeti da konfigurišete samo osnovnu autentifikaciju (HTTP Basic Auth) u kontrolerima. Optimalno, autentifikaciju i autorizaciju treba odraditi pre nego što zahtev pogodi neki endpoint.

Srećom, u Java web svetu postoji način da se upravo ovo uradi: mogu se definisati filteri ispred servleta, što znači da možemo razmisliti o pisanju SecurityFiltera i konfigurisati ga u Tomcat-u i na taj način filtrirati sve dolazne HTTP Request-ove pre nego što pogode aplikaciju.



U suštini, tok komunikacije je sledeći:

1. Prva stvar koja treba da se uradi jeste da se izvuku korisničko ime i lozinka iz zahteva. Kredencijali mogu biti poslati putem osnovnog Auth HTTP zaglavlja, tela HTTP zahteva, kolačića i sl.
2. Zatim se potvrđuje da je kombinacija korisničkog imena i lozinke ispravna (na primer, čita se iz baze podataka). Ukoliko to ne može da se potvrdi, zahtev se odbija, a klijentu koji je inicirao komunikaciju se vraća HTTP Response sa postavljenim status kodom 401 Unauthorized, u suprotnom se izvršava korak 3.
3. Nakon uspešne provere identiteta, vrši se proveri da li korisnik ima prava za pristup traženom endpoint-u, proverom uloge ili dozvola. Ukoliko korisnik nema prava pristupa, zahtev se odbija, a klijentu koji je inicirao komunikaciju se vraća HTTP Response sa postavljenim status kodom 403 Forbidden.

Ako zahtev uspešno zadovolji sve ove provere, tada se zahtev može propustiti dalje do traženog kontrolera!

Filter Chain

U aplikaciji, kod jednog filtera brzo postaje ogroman i težak za održavanje, jer se zahtev može na različite načine procesirati. U tom slučaju, najbolji pristup bi bio da se jedan filter razbije na više manjih, koji će se kasnije uvezati da rade zajedno - eng. Filter Chain. Upotrebom ovog principa, zahtev prolazi kroz čitav lanac filtera od prvog do poslednjeg u tačno definisanom redosledu. Svaki filter vrši proveru za koju je zadužen. Ukoliko

zahtev ne zadovoljava neki uslov filtera, zahtev se odbija i vraća se korisniku sa statusom o grešci, dok se u suprotnom prosleđuje sledećem filteru, sve dok se ne prosledi do Dispatcher Servlet-a.

Na primer, HTTP zahtev koji se šalju serveru aplikacije:

1. Prvo, prolazi kroz LoginMethodFilter
2. Zatim, prolazi kroz AuthenticationFilter
3. Zatim, prolazi kroz AuthorizationFilter
4. Konačno stiže do DispatcherServlet-a.
5.

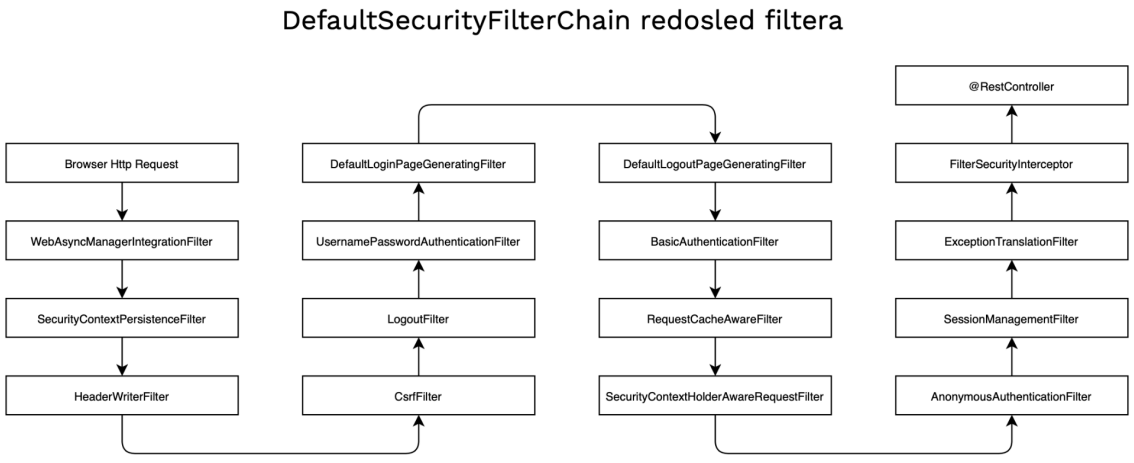
Ovaj koncept se naziva Filter Chain. Poslednja metoda u svakom filteru mora da delegira zahtev sledećem filteru u nizu:

```
chain.doFilter(request, response);
```

Pomoću ovako definisanog lanca filtera može sa implementirati svaki mehanizam autentifikacije ili autorizacije koji odgovara potrebama aplikacije, bez potrebe izmene prethodno implementiranih metoda!

Default Security Filter Chain

Kada se Spring Security ispravno integriše sa ostatkom aplikacije, on dodaje svojih 15 predefinisanih filtera u odgovarajućem redosledu. Kada HTTP Request stigne do Spring aplikacije, on će proći kroz svih ovih 15 filtera pre nego što konačno pogodi kontrolere. Redosled filtera je takođe važan, kao što je ranije napomenuto.



Objasnićemo samo par ključnih filtera. Detaljnije o filterima možete pročitati na linku ili pogledati Spring Security source code za detalje implementacije.

- **BasicAuthenticationFilter**: iz zahteva čita Basic Auth HTTP Header ako postoji. Iz tog zaglavlja se čitaju korisničko ime i lozinka i pokušava se autentifikacija korisnika. Kako postaviti Basic autentifikaciju, možete pogledati na linku.
- **UsernamePasswordAuthenticationFilter**: iz request parametara ili tela HTTP zahteva se čitaju korisničko ime i lozinka i pokušava se autentifikacija korisnika.
- **DefaultLoginPageGeneratingFilter**: generiše login stranicu. Mora se eksplicitno isključiti da se stranica ne bi generisala. Ukoliko ste dobili izgenerisanu login stranicu nakon uključivanja Spring Security-ja u projekat, to je zbog ovog filtera.
- **DefaultLogoutPageGeneratingFilter**: generiše logout stranicu. More se eksplicitno isključiti da se stranica ne bi generisala.
- **FilterSecurityInterceptor**: u ovom filteru se vrši proces autorizacije.

Ovi filteri su u suštini Spring Security i oni rade većinu posla. Na nama je da konfigurišemo način na koji rade svoj posao, npr. koje URL adrese treba zaštititi, koje ignorisati, koje tabele baza podataka koristiti za autentifikaciju, itd.

U nastavku je opisano kako se konfiguriše Spring Security.

Konfiguracija

WebSecurityConfigurerAdapter

Konfiguraciju Spring Security-ja ćemo izvršiti kroz klasu koja je:

1. Anotirana **@EnableWebSecurity** anotacijom i
2. Nasleđuje **WebSecurityConfigurerAdapter** koja u osnovi nudi DSL i metode za konfiguraciju. Pomoću ovih metoda možemo odrediti koje URL-ove u aplikaciji želimo zaštititi, kao i definisati ostale mehanizme za autentifikaciju.

Primer jednog WebSecurityConfigurerAdapter-a:

```
@Configuration
@EnableWebSecurity // (1)
public class WebSecurityConfig extends WebSecurityConfigurerAdapter { // (1)

  @Override
  protected void configure(HttpSecurity http) throws Exception {  // (2)
      http
        .authorizeRequests()
          .antMatchers("/", "/home").permitAll() // (3)
          .anyRequest().authenticated() // (4)
          .and()
       .formLogin() // (5)
          .loginPage("/login") // (5)
          .permitAll()
          .and()
      .logout() // (6)
        .permitAll()
        .and()
      .httpBasic(); // (7)
  }
}
```

1. dodavanje potrebnih anotacija i nasleđivanje klase
2. override-ovanjem ove metode, dobijamo DSL kojim konfigurišemo naš filter
3. svi zahtevi koji stižu na / i /home URL su dozvoljeni svima, što znači da korisnik ne mora biti autentifikovan da bi im pristupio. Koriste se antMatcher izrazi, koji takođe omogućavaju i upotrebu wildcard karaktera (*, **, ?)
4. za pristup svakom drugom URL-u u aplikaciji, korisnik prethodno mora biti uspešno autentifikovan
5. omogućavanje autentifikacije korisnika preko forme. Može se koristiti stranica koja je napravljena, a ne default-na koju generiše Spring Security, što se postavlja kroz loginPage("/login"). Svako mora da pristupi login stranici i zato se navodi permitAll()
6. ista priča kao i za login
7. dozvoljava se i HTTP Basic autentifikacija (postavljanje HTTP Basic Auth Header-a za autentifikaciju.)

Da sumiramo, u metodi configure(HttpSecurity http) definišemo koji endpoint-i su zaštićeni, a koji su javni:

- sa authenticated() označavamo URL-ove koji su zaštićeni (samo im autentifikovani korisnici mogu pristupiti),
- sa permitAll() označavamo URL-ove kojima svi mogu pristupati (što će sigurno biti metode za logovanje i registraciju).

U našem primeru, pogledati implementaciju klase **WebSecurityConfig** koja se nalazi u rs.ac.uns.ftn.informatika.spring.security.config paketu.

Implementacija autentifikacije

U bazi podataka se čuvaju informacije o korisnicima. Postoji tabela u kojoj se čuvaju korisničko ime, lozinka (ali ne u plain tekstu, već heširana) i proizvoljne dodatne informacije o korisnicima sistema. Moramo da konfigurišemo Spring Security-ju tako da zna u kojoj tabeli će da traži informacije o korisnicima prilikom procesa autentifikacije. Da bismo ovo postigli, moramo da definišemo dva bean-a:

1. **UserDetailsService** i
2. **PasswordEncoder**.

UserDetailsService

Interfejs koji ima samo jednu metodu loadUserByUsername koja vraća UserDetails objekat. Metoda kao parametar prima samo jednu vrednost - korisničko ime. U ovoj metodi se proverava da li korisnik postoji u sistemu i učitava se iz baze podataka. U primeru, pogledati implementaciju klase **CustomUserDetailsService** iz paketa rs.ac.uns.ftn.informatika.spring.security.service.impl.

Ovaj bean se uvodi u *SpringSecurity* konfiguraciju u klasi *WebSecurityConfig*, pomoću sledećeg koda:

```
@Autowired
private CustomUserDetailsService jwtUserDetailsService;
```

```
@Autowired
public void configureGlobal(AuthenticationManagerBuilder auth) throws Exception {
 auth.userDetailsService(jwtUserDetailsService).passwordEncoder(passwordEncoder());
}
```

```
.addFilterBefore(new TokenAuthenticationFilter(tokenUtils, jwtUserDetailsService), BasicAuthenticationFilter.class);
```

TokenAuthenticationFilter je filter kojji je definisan u paketu rs.ac.uns.ftn.informatika.spring.security.security.auth. Kod koji je iznad napisan dodaje ovaj filter u lanac pre BasicAuthenticationFilter-a. Filter će, kao i svaki drugi iz Default Security Filter Chain-a presresti svaki HTTP zahtev koji stiže od klijenta ka serverskoj strani. Ima jednu metodu **doFilterInternal** koja zapravo predstavlja kod koji se izvršava kada dođe na red u lancu. U primeru je definisano da se koristi autentifikacija bazirana na tokenima, i koristi se JWT token (objašnjeno u prezentaciji). Kreirana je **TokenUtils** klasa u paketu rs.ac.uns.ftn.informatika.spring.security.security u kojoj su implementirane metode za rad sa JWT tokenom. Ukoliko je token prosleđen, pripada korisniku koji postoji u sistemu i validan je, podaci o korisniku koji je vlasnik tokena se smeštaju u SecurityContextHolder, kako bi ti podaci dalje bili dostupni ostalim filterima (npr. za autorizaciju), ali i bilo gde u korisničkom kodu (npr. u metodama servisa ili kontrolera). Prilikom implementacije filtera, ne sme se zaboraviti da poslednja naredba **doFilterInternal** metode bude prosleđivanje HTTP zahteva sledećem filteru u lancu:

```
chain.doFilter(request, response);
```

PasswordEncoder

Pomoću ovog bean-a se definiše algoritam koji će se koristiti za heširanje lozinki. Prilikom autentifikacije, Spring Security automatski heširа vrednosti koje dolaze sa strane klijenta i poredi ih sa onim što piše u bazi podataka (što je pročitano pomoću **UserDetailsService** bean-a), zato treba voditi računa da se prilikom upisivanja korisnika u bazu podataka lozinke heširaju!

Ovaj bean se uvodi u SpringSecurity konfiguraciju u klasi **WebSecurityConfig**, pomoću sledećeg koda:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Autorizacija

Prethodno smo naveli primer web shop aplikacije koja razlikuje dva tipa korisnika: kupce i administratore. Neka je npr. URL aplikacije www.webshop.com a deo aplikacije za administraciju www.webshop.com/admin. Običan kupac ne sme da pristupi dalu aplikacije za administraciju. U suštini, želimo drugačija prava pristupa za različite korisnike na osnovu njihovih uloga i dozvola. Mehanizmi autentifikacije koji su do sada objašnjeni nam neće biti dovoljni da bismo ovo implementirati bez izmena već postojećih metoda.

GrantedAuthority

Spring Security poseduje sledeće pojmove (objedinjene pod pojmom GrantedAuthority):

- authority - predstavlja prava koja korisnik ima (fine grained permissions) i
- role - predstavlja ulogu koju korisnik ima, sa prefiksom ROLE_ koji Spring Security sam dodaje (coarse grained permissions).

Predstavljene su interfejsom **GrantedAuthority**.

U primeru, pogledati klasu Role iz paketa rs.ac.uns.ftn.informatika.spring.security.model, koja je ujedno anotirana i **@Entity** anotacijom. Klasa User iz modela ima listu roles, što znači da se one čuvaju u bazi. Role koje postoje u sistemu su u bazu dodati kroz skriptu za inicijalizaciju baze podataka data-postgres.sql.

```
INSERT INTO ROLE (name) VALUES ('ROLE_USER');
INSERT INTO ROLE (name) VALUES ('ROLE_ADMIN');
```

GrantedAuthority se koristi na nivou URL-ova da se dodatno zaštite. Koriste se za one URL-ove koji su vezani za tip korisnika, odnosno tamo gde autentifikacija nije dovoljna.

GrantedAuthority-je definisane iznad možemo koristiti na sledeće načine:

1. metoda **hasAuthority()** upotrebom Spring Security DSL-a:

```
http
    .authorizeRequests()
        .antMatchers("/admin").hasAuthority("ROLE_ADMIN") (1)
        .antMatchers("/shop").hasAnyAuthority("ROLE_ADMIN", "ROLE_USER") (2)
```

(1) za pristup ruti admin korisnik mora da bude autentifikovan i da ima authority "ROLE_ADMIN". (2) za pristup ruti_shop_korisnik mora da bude autentifikovan i da ima_authority_ili "ROLE_ADMIN" ili "ROLE_USER".

2. metoda **hasRole()** Spring Security DSL-a:

```
http
    .authorizeRequests()
        .antMatchers("/admin").hasRole("ADMIN") (1)
        .antMatchers("/shop").hasAnyRole("ADMIN", "USER") (2)
```

(1) za pristup ruti admin korisnik mora da bude autentifikovan i da ima authority "ROLE_ADMIN". Obratiti pažnju da se poziva_hasRole_metoda koja dodaje prefiks ROLE_i zato se uloge navode bez ovog prefiksa! (2) za pristup ruti shop korisnik mora da bude autentifikovan i da ima authority ili "ROLE_ADMIN" ili "ROLE_USER". Obratiti pažnju da se poziva hasAnyRole metoda koja dodaje prefiksROLE_ i zato se uloge navode bez ovog prefiksa!

3. upotrebom **@PreAuthorize** anotacije

Da ne bismo morali u WebSecurityConfig klasi da navodimo sve rute koje želimo dodatno da osiguramo, može se koristiti anotacija @PreAuthorize kojom se direktno anotiraju metode kontrolera. Sve napisano pod tačkama 1. i 2. ove sekcije važi za ovu anotaciju, samo se odgovarajuće metode pišu kao parametar anotacije.

Na primer, anotaciju možemo napisati na sledeći način:

```
@PreAuthorize("hasRole('USER')")
```

U primeru, pogledati klasu **UserController** iz paketa rs.ac.uns.ftn.informatika.spring.security.controller.

Da bi ova anotacija radila, mora se navesti **@EnableGlobalMethodSecurity(prePostEnabled = true)** u WebSecurityConfig klasi!

Pokretanje Spring aplikacije (Eclipse)

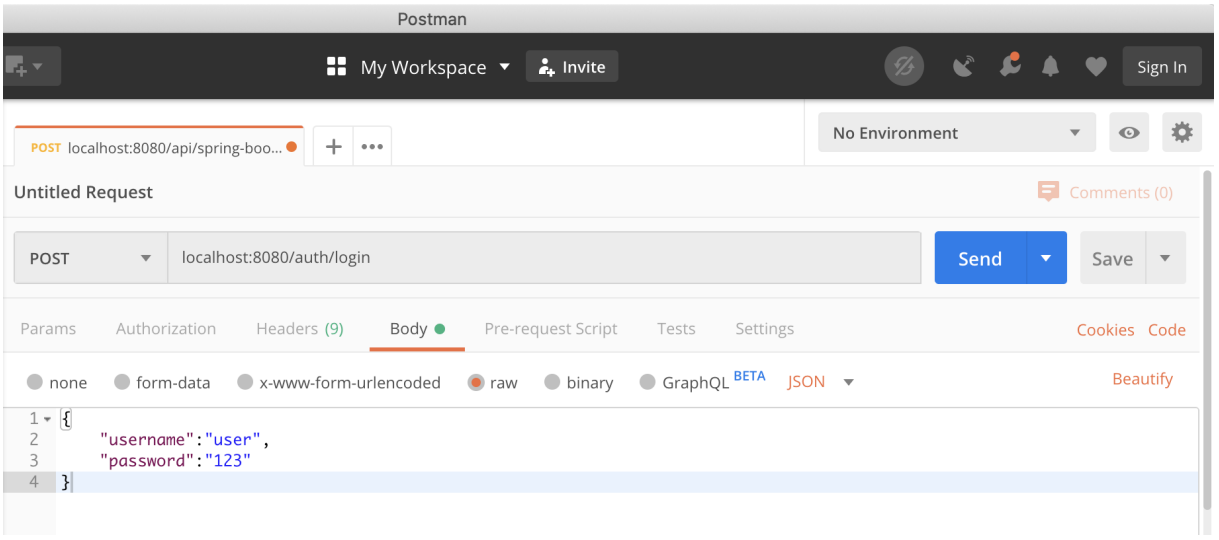
- importovati projekat u workspace: Import -> Maven -> Existing Maven Project
- instalirati sve dependency-je iz pom.xml
- desni klik na projekat -> Run as -> Java Application / Spring Boot app (ako je instaliran STS plugin sa Eclipse marketplace)

Pokretanje Angular aplikacije

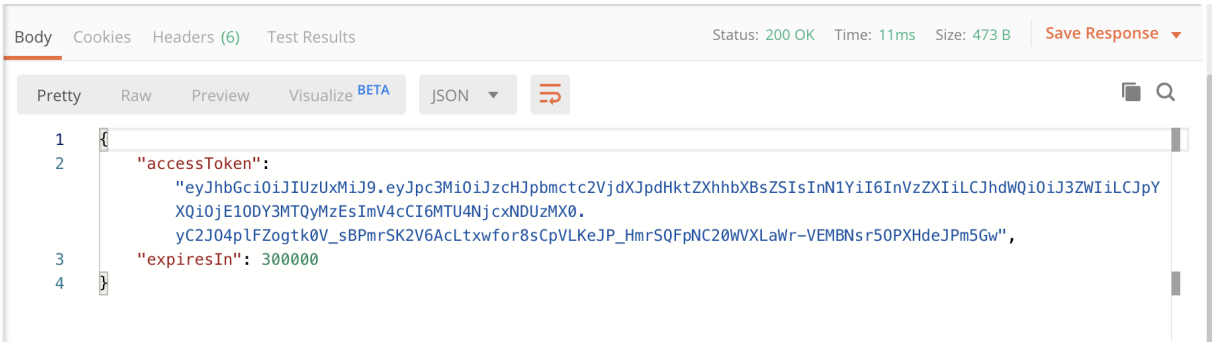
- Prerequisites
- preko komandne linije pozicionirati se u folder client i kucati naredbu: npm install, zatim ng serve

Napomena: za razumevanje Spring Security primera nije neophodno da pokrećete klijentsku aplikaciju. Svi potrebni koncepti se nalaze na serverskoj strani. Sve zahteve na Spring aplikaciju možete da šaljete preko Postman-a na sledeći način:

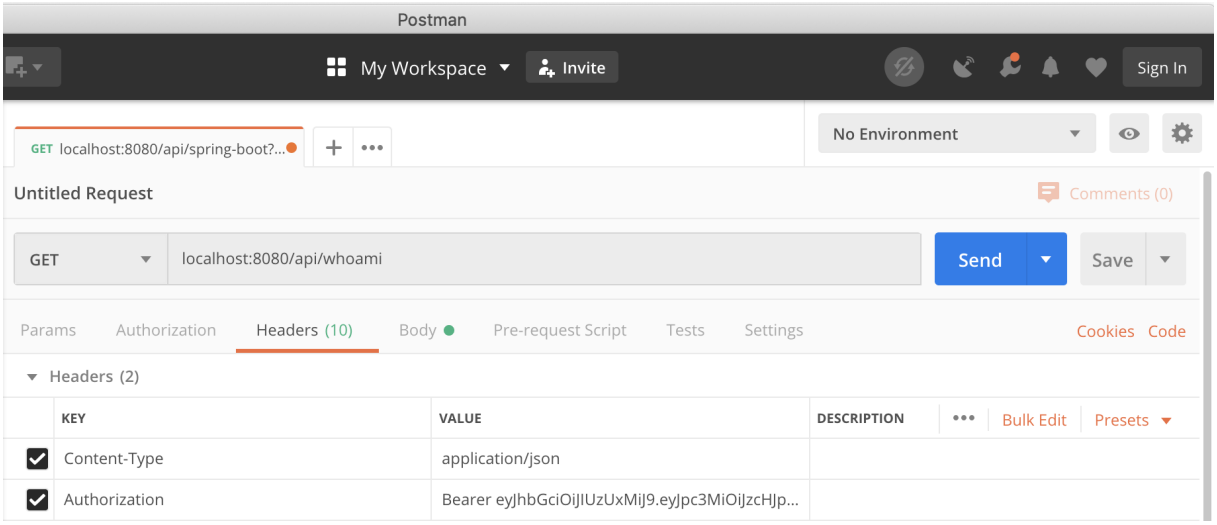
1. Poslati zahtev za logovanje. Kredencijali korisnika su sledeći - korisničko ime: user, lozinka: 123.



Kada je autentifikacija uspešna, login metoda vraća JWT token u accessToken atributu. Kopirati dobijeni token (kopirati ceo token, bez navodnika).



2. Kada šaljete zahteve na zaštećene URL-ove, obavezno poslati i token koji ste prethodno dobili. Token smeštate u zaglavlje zahteva pod ključem Authorization, jer serverska strana očekuje token u tom zaglavlju. Kao vrednost polja navodi se prefiks Bearer razmak pa JWT token kao na slici.



Ako je sve u redu, odgovor koji ste dobili od servera je sledeći:

